Machine Learning & Deep Learning

Interview Questions and Answers

Part 2 — Deep Learning & NLP

Fundamentals • Statistics • Classical ML • Evaluation • Deep Learning • CNNs • RNNs • Transformers • NLP • MLOps • Scenarios • Coding

Table of Contents

13. Deep Learning Fundamentals
14. Training Deep Networks & Regularization
15. Convolutional Neural Networks & Computer Vision
16. RNNs, LSTMs & Sequence Models
17. Attention & Transformers
18. Natural Language Processing

Study Roadmap: Beginner to Advanced

Within every section, questions go roughly from basic to advanced. Suggested order:

Level	Focus	Sections
1. Beginner	Core vocabulary, types of ML, basic statistics, data cleaning, linear/logistic regression, decision trees, basic metrics, neural network basics, Python libraries	1, 2 (first half), 3, 4, 6 (first half), 9 (first half), 13 (first half), 21 (metrics, pandas, linear regression)
2. Intermediate	Optimization, ensembles and boosting, SVM/KNN/Naive Bayes, clustering and PCA, tuning, time series, training deep nets, CNNs, RNNs, NLP	5, 6, 7, 8, 10, 11, 13–16, 18, 21 (k-means, KNN, logistic regression, PyTorch loop)
3. Advanced	Statistical testing and causal inference, XGBoost internals, calibration and uncertainty, recommender and ranking systems, Transformers, MLOps, system design, implementing attention/NMS from scratch	2 (second half), 9 (second half), 12, 17, 19, 20, 21 (attention, NMS, AUC)

Tip: For each answer, practise explaining it in 30 seconds, then be ready to go one level deeper (the math, a trade-off, or a real example from your projects).

13. Deep Learning Fundamentals

Q1. What is deep learning and why has it succeeded recently?

Deep learning uses neural networks with many layers to learn hierarchical representations directly from raw data, removing manual feature engineering. Success drivers: large datasets, GPUs/TPUs, better architectures (ResNets, Transformers), better training techniques (ReLU, BatchNorm, Adam, residual connections), and open-source frameworks.

Q2. Explain the structure of a neural network. What are weights and biases?

An input layer, one or more hidden layers, and an output layer. Each neuron computes a weighted sum of its inputs plus a bias and applies an activation: `a = f(w·x + b)`. **Weights** set the strength of each connection; the **bias** shifts the activation threshold so the decision boundary need not pass through the origin. In dense (fully connected) layers every neuron connects to all neurons of the previous layer. Training adjusts weights and biases via backpropagation.

Q3. What is a tensor? Why are GPUs used for deep learning?

A tensor is an n-dimensional array: scalar (0-D), vector (1-D), matrix (2-D), and higher, e.g., a batch of images has shape (N, C, H, W). Deep learning is dominated by large matrix multiplications and convolutions that are highly parallel; GPUs have thousands of cores and high memory bandwidth to do these in parallel, often 10–100× faster than CPUs. TPUs and NPUs are chips specialized for the same operations.

Q4. What is a perceptron? Why can't it solve XOR?

A single neuron computing step(w·x + b); a linear classifier. XOR is not linearly separable, so one perceptron can't solve it. A multi-layer perceptron with a hidden layer and non-linear activation can.

Q5. What is the Universal Approximation Theorem?

A feed-forward network with a single hidden layer of enough neurons and a non-linear activation can approximate any continuous function on a compact domain to arbitrary accuracy. It says nothing about how many neurons are needed, or whether training can find the weights; deep networks represent many functions far more efficiently than wide shallow ones.

Q6. Why do we need non-linear activation functions?

Without them, stacked linear layers collapse into a single linear transformation ($W_2W_1x = Wx$), so depth adds no expressive power.

Q7. Compare common activation functions.

Activation	Formula / Range	Notes
Sigmoid	$1/(1+e^{-x})$, (0,1)	Saturates → vanishing gradients; not zero-centered; used for binary outputs and gates.
Tanh	(−1,1)	Zero-centered, still saturates; used in RNNs.
ReLU	max(0,x)	Cheap, no saturation for x>0, sparse; "dying ReLU" for x<0.
Leaky ReLU / PReLU	max(αx, x)	Small slope for negatives fixes dying ReLU; PReLU learns α.
ELU / SELU	smooth negative part	Mean closer to zero; SELU self-normalizes.
GELU	x·Φ(x)	Smooth; default in BERT/GPT.
Swish / SiLU	x·σ(x)	Smooth, non-monotonic; EfficientNet, LLaMA (SwiGLU).
Softmax	$e^{z_i}/\Sigma e^{z_j}$	Multi-class probability output.

Q8. What is the dying ReLU problem?

If a neuron's pre-activation is negative for all inputs (e.g., due to a large learning rate or bad bias), its gradient is zero and it never updates again. Fix with Leaky ReLU/ELU/GELU, proper initialization, and lower learning rates.

Q9. Which activation should be used in the output layer (sigmoid vs softmax vs linear)?

- Binary classification: 1 sigmoid unit + binary cross-entropy.
- Multi-class (one label per sample): softmax over K units + categorical cross-entropy; probabilities sum to 1.
- Multi-label (several labels per sample): K independent sigmoid units + binary cross-entropy per label.
- Regression: linear (no activation); ReLU/softplus if output must be non-negative.

Hidden layers typically use ReLU/GELU, not sigmoid.

Q10. Explain forward propagation and backpropagation.

Forward: compute layer outputs sequentially to get predictions and loss. Backward: apply the chain rule from the loss back through each layer to compute gradients of the loss w.r.t. every parameter, reusing intermediate results (reverse-mode automatic differentiation), then update weights with an optimizer. Cost of backward pass is about 2× forward.

Q11. Derive the gradient of softmax + cross-entropy.

For logits z, p = softmax(z), and one-hot target y, $L = -\Sigma y_k \log p_k$. Then $\partial L/\partial z_k = p_k - y_k$. This simple, well-behaved gradient is why softmax and cross-entropy are combined (also numerically stable via log-sum-exp). Similarly, sigmoid + BCE gives p − y.

Q12. What is automatic differentiation? Static vs dynamic computation graphs?

Autodiff computes exact derivatives by recording elementary operations in a computational graph and applying the chain rule (reverse mode for many-inputs-to-scalar-loss). Dynamic graphs (PyTorch eager) are built on the fly, easy to debug, support Python control flow. Static graphs (TF1, compiled modes like `torch.compile`, JAX jit, XLA) allow global optimizations.

Q13. What are vanishing and exploding gradients? How to address them?

During backprop, gradients are products of many layer Jacobians; if factors are <1 they shrink exponentially (early layers stop learning), if >1 they explode (unstable training, NaNs). Remedies: ReLU-family activations, proper initialization (Xavier/He), batch/layer normalization, residual connections, LSTM/GRU gating, gradient clipping, smaller learning rates.

Q14. Why does weight initialization matter? Explain Xavier and He initialization.

All-zero initialization makes neurons symmetric (they learn the same thing). Too large/small weights cause exploding/vanishing activations. **Xavier/Glorot**: Var(w) = 2/(fan_in + fan_out), designed for tanh/sigmoid. **He/Kaiming**: Var(w) = 2/fan_in, designed for ReLU (which zeroes half the inputs). Both keep activation variance roughly constant across layers.

Q15. Which loss functions are used for which tasks?

- Regression: MSE, MAE, Huber, log-cosh, quantile (pinball) loss.
- Binary classification: binary cross-entropy (BCEWithLogits), hinge.
- Multi-class: categorical cross-entropy; multi-label: per-label sigmoid BCE.
- Imbalanced/detection: focal loss = $-\alpha(1-p)^{\gamma}\log p$ (down-weights easy examples).
- Segmentation: Dice loss, IoU/Jaccard loss, combined with CE.
- Metric learning: contrastive loss, triplet loss (anchor-positive closer than anchor-negative by margin), InfoNCE.
- Distributions: KL divergence; GANs: adversarial loss; ranking: pairwise/listwise losses.

Q16. What is the difference between a logit, a probability and log-odds?

Logits are raw unnormalized scores output by the last layer. Softmax/sigmoid convert logits into probabilities. For binary sigmoid output, the logit equals log-odds $\log(p/(1-p))$. Losses like `CrossEntropyLoss` take logits for numerical stability.

Q17. What are the differences between ML and DL?

Classical ML relies on hand-engineered features and works well on small/structured data with fast training and better interpretability. DL learns features automatically, excels on unstructured data (images, text, audio) with lots of data and compute, but is less interpretable and more resource intensive.

Q18. How do you count parameters in dense and convolutional layers?

Dense: (inputs × outputs) + outputs (bias). Conv2D: $(k_h \times k_w \times C_{in} + 1) \times C_{out}$. Example: 3×3 conv, 64→128 channels = $(3 \cdot 3 \cdot 64 + 1) \cdot 128 = 73{,}856$.

Q19. How do you choose the number of hidden layers and neurons?

There is no formula. Start from a proven architecture for the data type, or a small network (1–3 hidden layers) for tabular data. Increase capacity until training error is low and validation stops improving, then control overfitting with regularization; a slightly larger network with dropout/weight decay usually beats a too-small one. Tune with random or Bayesian search and prefer transfer learning when possible.

Q20. What is an autoencoder and what is it used for?

A network with an encoder that compresses the input into a low-dimensional latent code (bottleneck) and a decoder that reconstructs the input; trained to minimize reconstruction loss without labels. Uses: non-linear dimensionality reduction, denoising (denoising autoencoder), anomaly detection (high reconstruction error flags anomalies), feature learning and pretraining. Unlike PCA, it can learn non-linear structure.

Q21. When would you use an MLP vs CNN vs RNN vs Transformer?

- **MLP**: fixed-size feature vectors / tabular data (though gradient boosting often wins there).
- **CNN**: grid-like data with local patterns: images, spectrograms, 1D signals.
- **RNN/LSTM/GRU**: sequential data with small datasets, streaming or low-latency step-by-step processing, some time series.
- **Transformer**: text and sequences with long-range dependencies and large data; ViT for images with lots of data.

14. Training Deep Networks & Regularization

Q22. How do you prevent overfitting in neural networks?

Get more data or use data augmentation; use transfer learning; add regularization (weight decay/L2, dropout, label smoothing); use batch normalization; apply early stopping on validation loss; reduce model size; use ensembles; and make sure train/validation splits are clean (no leakage or duplicates).

Q23. Explain batch normalization. Why does it help?

Normalizes each feature over the mini-batch to zero mean and unit variance, then applies learnable scale γ and shift β. Uses running averages at inference. Benefits: allows higher learning rates, faster convergence, less sensitivity to initialization, mild regularization. Originally attributed to reducing "internal covariate shift"; later work suggests it mainly smooths the loss landscape. Downsides: depends on batch size, differs between train/eval, awkward for RNNs and small batches.

Q24. Batch norm vs layer norm vs instance norm vs group norm?

BatchNorm: normalize across batch per channel (CNNs, large batches). **LayerNorm**: normalize across features per sample, batch-independent (Transformers, RNNs). **InstanceNorm**: per sample per channel (style transfer). **GroupNorm**: per sample over groups of channels (small-batch detection/segmentation). **RMSNorm**: LayerNorm without mean-centering (LLaMA), cheaper.

Q25. How does dropout work? Why does it regularize?

During training, randomly zero each activation with probability p and scale survivors by $1/(1-p)$ (inverted dropout) so inference needs no change. It prevents co-adaptation of neurons and approximates averaging an exponential ensemble of sub-networks. Disabled at inference (`model.eval()`). MC Dropout keeps it on at inference to estimate uncertainty.

Q26. L2 regularization vs weight decay?

For plain SGD they are equivalent. For adaptive optimizers like Adam they differ, because L2's gradient is rescaled per parameter; decoupled weight decay (AdamW) applies shrinkage directly and works better.

Q27. What is label smoothing?

Replace one-hot targets with $(1-\varepsilon)$ for the true class and $\varepsilon/(K-1)$ (or ε/K) for others. Prevents over-confident logits, improves calibration and generalization (used in Inception, Transformers).

Q28. What are residual (skip) connections and why do they work?

Output = F(x) + x. The layer learns a residual, so learning identity is easy; gradients flow directly through the identity path, mitigating vanishing gradients and the degradation problem, enabling 100+ layer networks (ResNet) and deep Transformers.

Q29. Describe a typical training loop (e.g., in PyTorch).

1. For each epoch, call `model.train()` and iterate over mini-batches from a DataLoader.
2. Move data to the device; forward pass to get outputs; compute loss.
3. `optimizer.zero_grad()` (gradients accumulate otherwise), `loss.backward()` , optionally clip gradients, `optimizer.step()` , step the LR scheduler.
4. Validation: `model.eval()` (disables dropout, uses BatchNorm running stats) inside `torch.no_grad()` (no gradient tracking, saves memory); compute metrics.
5. Log metrics, checkpoint the best model, apply early stopping.

Q30. PyTorch vs TensorFlow/Keras?

PyTorch: Pythonic, eager (dynamic graphs), easy debugging, dominant in research and increasingly in production (TorchScript, `torch.compile`, ONNX). **TensorFlow/Keras**: high-level Keras API for quick model building, mature production ecosystem (TF Serving, TF Lite for mobile, TF.js), graph compilation with `tf.function`. Both support GPUs/TPUs and distributed training; the choice is often driven by team and deployment ecosystem.

Q31. What are the steps to debug a neural network that isn't learning?

1. Overfit a tiny batch first; if it can't reach ~0 loss, there's a bug.
2. Check data pipeline: labels aligned, normalization, augmentation correctness, shuffling.
3. Check loss/activation pairing (logits vs probabilities), loss scale at init (~ln K for K classes).
4. Tune learning rate (LR range test); check for NaNs, exploding/vanishing gradients (monitor gradient norms).
5. Verify `model.train()` / `eval()`, zeroing gradients, correct optimizer params.
6. Simplify architecture; then add complexity back.

Q32. What is mixed-precision training?

Use FP16/BF16 for most computations and FP32 master weights for updates. Speeds up training on Tensor Cores and halves memory. FP16 needs loss scaling to avoid gradient underflow; BF16 has FP32's exponent range so usually doesn't.

Q33. What is gradient accumulation and gradient checkpointing?

Gradient accumulation: sum gradients over several small batches before an update to emulate a large batch under memory limits. **Gradient (activation) checkpointing**: store only some activations during forward and recompute the rest during backward, trading compute for memory.

Q34. How is deep learning training distributed across devices?

Data parallelism (DDP): replicate model, split batches, all-reduce gradients. **ZeRO / FSDP**: shard optimizer states, gradients and parameters across devices. **Tensor parallelism**: split individual layers' matrices. **Pipeline parallelism**: split layers across devices with micro-batches. Large LLMs combine all (3D parallelism).

Q35. What is knowledge distillation?

A small student model is trained to match the softened output distribution (soft targets, temperature T) of a large teacher, plus the true labels. Soft targets carry "dark knowledge" about class similarities. Used to compress models (DistilBERT, TinyLLMs).

15. Convolutional Neural Networks & Computer Vision

Q36. Why use CNNs instead of fully connected networks for images?

Local connectivity (neurons see local patches), **weight sharing** (same filter across positions → far fewer parameters), and **translation equivariance** (features detected anywhere). Pooling adds some translation invariance. Hierarchy: edges → textures → parts → objects.

Q37. How does a convolution operation work? What is a feature map?

A small learnable filter (kernel, e.g., $3\times3\times C_{in}$) slides over the input; at each position it computes the sum of element-wise products plus a bias, producing one value of the output **feature map**. Each filter yields one output channel, so N filters give N feature maps. Early filters learn edges and colors; deeper filters learn textures, parts and objects. Convolution is typically followed by an activation (ReLU) and sometimes pooling.

Q38. What is the output size formula for a convolution?

Output = $\lfloor (W - K + 2P)/S \rfloor + 1$ where W = input size, K = kernel size, P = padding, S = stride. "Same" padding with stride 1 uses $P = (K-1)/2$.

Q39. What are padding, stride, and dilation?

Padding: add border pixels to preserve spatial size and edge information. **Stride**: step size of the filter; stride > 1 downsamples. **Dilation**: spaces kernel elements to enlarge the receptive field without extra parameters (atrous convolution, used in DeepLab, WaveNet).

Q40. What is the receptive field?

The region of the input that affects a particular feature. It grows with depth, kernel size, stride, pooling and dilation. Deep layers need large receptive fields to capture global context.

Q41. Max pooling vs average pooling vs global average pooling?

Max pooling keeps the strongest activation (feature presence), average pooling smooths. Global average pooling averages each feature map into one value, replacing large FC layers, reducing parameters and overfitting. Strided convolutions can replace pooling.

Q42. What is a 1×1 convolution used for?

Mixes channels at each pixel: reduces/increases channel dimension cheaply (bottlenecks in Inception/ResNet), adds non-linearity, and acts as a per-pixel fully connected layer.

Q43. What are depthwise separable convolutions?

Split a standard conv into a depthwise conv (one filter per input channel) and a pointwise 1×1 conv. Cost drops by roughly a factor of $1/C_{out} + 1/K^2$ (8–9× fewer FLOPs for 3×3). Used in MobileNet, Xception, EfficientNet.

Q44. Summarize key CNN architectures.

- **LeNet-5** (1998): digit recognition; conv + pooling + FC.
- **AlexNet** (2012): ReLU, dropout, GPUs, data augmentation; ImageNet breakthrough.
- **VGG**: stacks of 3×3 convs; simple, deep, many parameters.
- **GoogLeNet/Inception**: parallel multi-scale filters with 1×1 bottlenecks.
- **ResNet**: residual connections; 50–152+ layers; bottleneck blocks.
- **DenseNet**: each layer connected to all previous layers (feature reuse).
- **MobileNet / ShuffleNet**: efficient mobile models.
- **EfficientNet**: compound scaling of depth, width, resolution.
- **ConvNeXt**: modernized CNN competitive with ViTs.

Q45. Why stack two 3×3 convolutions instead of one 5×5?

Same receptive field (5×5), fewer parameters ($18C^2$ vs $25C^2$), and an extra non-linearity, increasing expressiveness.

Q46. How do you apply transfer learning / fine-tuning in vision?

Load a pretrained backbone, replace the classification head. Small dataset similar to source: freeze backbone, train head. Larger or different dataset: unfreeze top layers or all, with a small learning rate (discriminative learning rates: lower for early layers). Use same input normalization as pretraining.

Q47. Image classification vs object detection vs semantic, instance and panoptic segmentation?

Classification: one label per image. Detection: bounding boxes + classes. Semantic segmentation: class per pixel (no instance separation). Instance segmentation: per-object masks. Panoptic: both semantic stuff and instance things.

Q48. Explain the R-CNN family.

R-CNN: selective search proposals → CNN on each crop → SVM; very slow. **Fast R-CNN**: run CNN once, RoI pooling on feature map, joint classification + box regression. **Faster R-CNN**: Region Proposal Network with anchors replaces selective search; end-to-end. **Mask R-CNN**: adds mask branch and RoIAlign (no quantization) for instance segmentation.

Q49. One-stage vs two-stage detectors? Explain YOLO.

Two-stage (Faster R-CNN): propose regions, then classify; more accurate, slower. One-stage (YOLO, SSD, RetinaNet): predict boxes and classes densely in one pass; real-time. YOLO divides the image into a grid; each cell predicts boxes, objectness, and class probabilities; later versions add anchors, multi-scale feature pyramids, and anchor-free heads. RetinaNet introduced focal loss to handle extreme foreground/background imbalance.

Q50. What are anchor boxes, IoU and Non-Max Suppression?

Anchors: predefined boxes of various scales/aspect ratios; the model predicts offsets from them. **IoU**: area of overlap / area of union between boxes. **NMS**: sort detections by confidence, keep the best, remove others with IoU above a threshold, repeat. Soft-NMS decays scores instead of removing.

Q51. How is mAP computed?

For each class, rank detections by confidence; a detection is TP if IoU ≥ threshold with an unmatched ground truth. Compute the precision-recall curve and its area (AP). mAP averages AP over classes (and over IoU thresholds 0.5:0.95 in COCO).

Q52. What is a Feature Pyramid Network (FPN)?

Combines high-resolution, low-level features with low-resolution, semantically strong features through a top-down pathway with lateral connections, giving strong multi-scale features for detecting small and large objects.

Q53. Explain U-Net and FCN for segmentation.

FCN: replaces FC layers with convs and upsamples to produce per-pixel predictions. **U-Net**: symmetric encoder-decoder with skip connections concatenating encoder features to decoder layers, preserving fine spatial detail; popular in medical imaging and diffusion models. Upsampling via transposed convolution or interpolation + conv. DeepLab uses atrous convolution + ASPP.

Q54. What is a transposed convolution? What are checkerboard artifacts?

A learnable upsampling operation (sometimes miscalled deconvolution) that maps a small input to a larger output. Uneven overlap when kernel size isn't divisible by stride causes checkerboard artifacts; fix with resize (nearest/bilinear) + conv.

Q55. What is a Vision Transformer (ViT)? ViT vs CNN?

ViT splits an image into patches (e.g., 16×16), linearly embeds them, adds positional embeddings and a [CLS] token, and feeds them to a Transformer encoder. Global attention from the first layer; weaker inductive bias, so needs large datasets or strong augmentation/distillation (DeiT). Swin Transformer uses shifted local windows for hierarchical features and linear complexity. CNNs are more data-efficient; ViTs scale better with data.

Q56. What is Grad-CAM?

A visualization technique that weights the last convolutional feature maps by the gradient of the class score (global-average-pooled) to produce a heatmap of image regions important for the prediction.

Q57. What are face recognition / Siamese networks?

Siamese networks share weights across two (or three) inputs to learn an embedding where similar items are close. Trained with contrastive or triplet loss (FaceNet) or margin-based softmax (ArcFace). Enables one-shot verification by comparing embedding distances.

16. RNNs, LSTMs & Sequence Models

Q58. How does a recurrent neural network work?

Processes sequences step by step, maintaining a hidden state: $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$. Weights are shared across time steps. Configurations: one-to-many (captioning), many-to-one (sentiment), many-to-many (tagging, translation).

Q59. What is Backpropagation Through Time (BPTT)? Truncated BPTT?

Unroll the RNN over time and backpropagate through the unrolled graph, summing gradients for shared weights. Truncated BPTT limits backprop to a fixed number of steps for efficiency and stability.

Q60. Why do vanilla RNNs struggle with long-term dependencies?

Gradients are multiplied by the recurrent weight Jacobian at every step, so they vanish or explode exponentially with sequence length, making long-range dependencies hard to learn.

Q61. Explain the LSTM architecture.

LSTM adds a cell state c_t (a "conveyor belt") with gates:

Forget gate $f = \sigma(W_f[h_{t-1}, x_t])$ — what to discard.

Input gate $i = \sigma(...)$ and candidate $\tilde{c} = \tanh(...)$ — what to add.

$c_t = f \odot c_{t-1} + i \odot \tilde{c}$.

Output gate $o = \sigma(...)$; $h_t = o \odot \tanh(c_t)$.

The additive cell update lets gradients flow over many steps, mitigating vanishing gradients.

Q62. GRU vs LSTM?

GRU merges forget and input gates into an update gate, uses a reset gate, and has no separate cell state. Fewer parameters, faster, often similar performance; LSTM can be better on longer/more complex sequences.

Q63. What are bidirectional and stacked RNNs?

Bidirectional RNNs run one RNN forward and one backward and concatenate states, giving each position past and future context (good for tagging, not for autoregressive generation). Stacked (deep) RNNs feed one layer's hidden states as input to the next.

Q64. Explain the seq2seq encoder-decoder model.

An encoder RNN compresses the input sequence into a context vector; a decoder RNN generates the output sequence conditioned on it. The fixed-size bottleneck hurts long sequences, motivating attention (Bahdanau, Luong), where the decoder attends to all encoder states at each step.

Q65. What is teacher forcing and exposure bias?

Teacher forcing feeds the ground-truth previous token to the decoder during training, speeding convergence. At inference the model consumes its own predictions, so errors compound (exposure bias). Mitigations: scheduled sampling, sequence-level training.

Q66. Greedy decoding vs beam search?

Greedy picks the most probable token each step (fast, can miss better sequences). Beam search keeps the top-B partial sequences at each step, usually with length normalization; better for translation/summarization, but can be repetitive/bland for open-ended generation where sampling is preferred.

Q67. Why did Transformers replace RNNs?

RNNs process tokens sequentially (no parallelism across time), struggle with long dependencies, and are slow to train. Transformers use self-attention to connect all positions directly (path length O(1)), fully parallelize training, and scale better with data and compute.

17. Attention & Transformers

Q68. What is the attention mechanism?

A way to compute a weighted sum of values, where weights reflect relevance of each key to a query. It lets a model focus on relevant parts of the input dynamically instead of compressing everything into a fixed vector.

Q69. Explain scaled dot-product self-attention.

Each token embedding X is projected into $Q = XW_Q$, $K = XW_K$, $V = XW_V$.

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$.

Every token attends to every other token, producing context-aware representations.

Q70. Why divide by $\sqrt{d_k}$?

For random vectors with unit-variance components, the dot product's variance grows with d_k. Large logits push softmax into saturated regions with tiny gradients. Scaling keeps variance ~1.

Q71. What is multi-head attention and why use it?

Run h attention operations in parallel with different learned projections of smaller dimension (d_{model}/h), concatenate, and project. Each head can capture different relationships (syntax, coreference, position) at roughly the same cost as one full-size head.

Q72. Describe the original Transformer architecture.

Encoder block: multi-head self-attention → add & norm → position-wise feed-forward (two linear layers with activation, typically 4× hidden size) → add & norm. **Decoder** block: masked self-attention → cross-attention over encoder outputs → feed-forward, each with residual + norm. Inputs: token embeddings + positional encodings. Output: linear + softmax over vocabulary.

Q73. Why are positional encodings needed? What types exist?

Self-attention is permutation-invariant, so order must be injected. Types: sinusoidal (fixed, can extrapolate somewhat), learned absolute embeddings (BERT, GPT-2), relative position encodings (T5 bias), RoPE (rotary; rotates Q/K by position-dependent angles so dot products depend on relative position; LLaMA), ALiBi (linear distance penalty on attention scores; good extrapolation).

Q74. What is masking in Transformers?

Padding mask: ignore pad tokens. **Causal (look-ahead) mask**: in decoders, set scores for future positions to $-\infty$ so each token only attends to previous tokens, enabling autoregressive training in parallel.

Q75. Self-attention vs cross-attention?

Self-attention: Q, K, V come from the same sequence. Cross-attention: Q from one sequence (decoder), K and V from another (encoder output, image features, or text conditioning in diffusion models).

Q76. What is the computational complexity of self-attention, and how is it reduced?

$O(n^2 \cdot d)$ time and $O(n^2)$ memory in sequence length n. Reductions: sparse/local attention (Longformer, BigBird), low-rank/linear attention (Linformer, Performer), sliding windows, FlashAttention (IO-aware exact attention computed in tiles, avoiding materializing the n×n matrix), multi-query and grouped-query attention (fewer K/V heads to shrink KV cache).

Q77. Walk through the tensor shapes in multi-head attention. How many parameters does a Transformer layer have?

Input X: (B, n, d_{model}). Project to Q, K, V and split into h heads: (B, h, n, d_k) with $d_k = d_{model}/h$. Scores QK^T: (B, h, n, n) — the n^2 term. Weights × V: (B, h, n, d_k); concatenate heads back to (B, n, d_{model}) and apply W_O. Parameters per layer (ignoring biases and norms): attention $4 \cdot d_{model}^2$ (W_Q, W_K, W_V, W_O) + feed-forward $2 \cdot d_{model} \cdot d_{ff} = 8 \cdot d_{model}^2$ when $d_{ff} = 4 \cdot d_{model}$, so about $12 \cdot d_{model}^2$ per layer.

Q78. Encoder-only vs decoder-only vs encoder-decoder models?

Type	Examples	Pretraining	Best for
Encoder-only	BERT, RoBERTa, DeBERTa	Masked language modeling (bidirectional)	Classification, NER, embeddings, retrieval
Decoder-only	GPT, LLaMA, Mistral	Causal next-token prediction	Text generation, chat, general LLMs
Encoder-decoder	T5, BART	Span corruption / denoising	Translation, summarization

Q79. Explain BERT pretraining.

Masked Language Modeling: mask 15% of tokens (of these 80% [MASK], 10% random, 10% unchanged) and predict them using bidirectional context. **Next Sentence Prediction**: predict if sentence B follows A (later shown to be of little value; RoBERTa dropped it and trained longer with dynamic masking). Uses [CLS] for sentence-level tasks and [SEP] between segments; WordPiece tokenization.

Q80. What is the role of the feed-forward network in a Transformer block?

Applied independently to each position, it adds non-linear transformation capacity and holds most parameters; it is thought to store factual "key-value memories." Variants: GeLU FFN, SwiGLU (gated), and Mixture-of-Experts FFNs.

18. Natural Language Processing

Q81. What are the common text preprocessing steps?

Lowercasing, tokenization, removing punctuation/stop words/HTML, handling numbers and emojis, stemming or lemmatization, spelling normalization. For modern Transformers, minimal preprocessing is used: the model's own subword tokenizer handles raw text.

Q82. Stemming vs lemmatization?

Stemming chops suffixes heuristically ("studies" → "studi"; Porter, Snowball); fast but crude. Lemmatization uses vocabulary and POS to return dictionary forms ("studies" → "study", "better" → "good"); more accurate, slower.

Q83. Explain Bag-of-Words and TF-IDF.

BoW represents a document by word counts, ignoring order. TF-IDF weights term frequency by inverse document frequency: tfidf = tf(t,d) × log(N/df(t)), down-weighting common words. Both are sparse and lose semantics/order; n-grams capture some local order. Still strong baselines with logistic regression/Naive Bayes; BM25 is a TF-IDF-like ranking function for search.

Q84. What are word embeddings and why are they better than one-hot encoding?

One-hot vectors are sparse, vocabulary-sized, and every pair of words is equally distant, so no meaning is captured. Word embeddings are dense, low-dimensional (e.g., 100–1024) vectors learned so that words used in similar contexts are close together, capturing semantic and syntactic relationships, generalizing better and using less memory.

Q85. Explain word2vec (CBOW and Skip-gram).

Learns dense word vectors from context. **CBOW** predicts a word from surrounding words (faster, good for frequent words). **Skip-gram** predicts context words from the center word (better for rare words). Training efficiency via negative sampling (distinguish true context pairs from random pairs) or hierarchical softmax. Embeddings capture analogies (king − man + woman ≈ queen).

Q86. word2vec vs GloVe vs FastText?

word2vec: predictive, local context windows. GloVe: factorizes global word co-occurrence statistics (log counts). FastText: represents words as sums of character n-grams, so it handles out-of-vocabulary words and morphology. All produce static embeddings (one vector per word regardless of context).

Q87. Static vs contextual embeddings?

Static embeddings give "bank" the same vector in "river bank" and "bank account". Contextual embeddings (ELMo, BERT, GPT) compute token representations depending on the whole sentence, handling polysemy.

Q88. What is cosine similarity and why is it used for text?

`cos θ = A·B / (||A|| ||B||)`, ranging from −1 to 1 (0 to 1 for non-negative vectors like TF-IDF). It measures direction rather than magnitude, so document length doesn't dominate. Used for document similarity, semantic search, deduplication and recommendations. For L2-normalized vectors it equals the dot product.

Q89. Explain subword tokenization: BPE, WordPiece, SentencePiece.

Subwords balance vocabulary size and OOV handling. **BPE**: start from characters/bytes and iteratively merge the most frequent adjacent pair (GPT uses byte-level BPE). **WordPiece**: merges pairs that maximize training-data likelihood (BERT; "##" continuation prefix). **Unigram LM**: starts large and prunes tokens by likelihood. **SentencePiece**: library that treats text as raw stream including spaces (_), language-independent, supports BPE and unigram (T5, LLaMA).

Q90. What is perplexity?

exp of the average negative log-likelihood per token; the effective number of choices the model is uncertain among. Lower is better; only comparable across models with the same tokenizer/vocabulary.

Q91. What are common NLP tasks?

Text classification/sentiment, Named Entity Recognition, POS tagging, dependency parsing, question answering (extractive/generative), summarization (extractive/abstractive), machine translation, natural language inference, semantic similarity, coreference resolution, topic modeling, information extraction, text generation.

Q92. How is NER modeled?

As token classification with BIO/BIOES tags (B-PER, I-PER, O). Classic: CRF or BiLSTM-CRF (CRF enforces valid tag transitions). Modern: fine-tuned BERT-style token classifiers or LLM prompting. Evaluate with entity-level precision/recall/F1.

Q93. Explain BLEU and ROUGE.

BLEU: geometric mean of modified n-gram precisions (1–4) against references with a brevity penalty; for translation. **ROUGE**: recall-oriented n-gram (ROUGE-N) or longest common subsequence (ROUGE-L) overlap; for summarization. Both miss semantics/paraphrases; complemented by BERTScore, COMET, or human/LLM judges.

Q94. How would you build a text classifier today?

Baseline TF-IDF + logistic regression; then fine-tune a pretrained encoder (e.g., DeBERTa/RoBERTa) with a classification head; or use sentence embeddings + light classifier; or zero/few-shot LLM prompting if labeled data is scarce. Handle imbalance, evaluate with F1, and analyze errors.

Q95. How are sentence embeddings created and used?

Sentence-BERT fine-tunes a Siamese BERT with contrastive/NLI objectives and mean pooling so cosine similarity reflects semantic similarity. Used for semantic search, clustering, deduplication, and RAG retrieval. Bi-encoders are fast (precompute embeddings); cross-encoders jointly encode pairs and are more accurate for re-ranking.